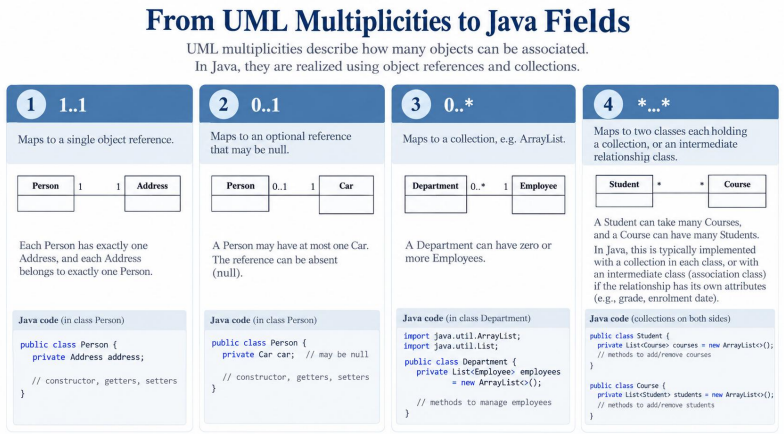
CSC241 — Object-Oriented Programming

Lecture 12 — Forward and Reverse Engineering

Translating UML class diagrams into Java code and recovering UML from existing Java programs



Dr Muhammad Shahid Bhatti
Assistant Professor
Department of Computer Science
Office H-17
GitHub: github.com/mshahidbhatti/CSC241-OOP

UML → Java mapping | Java → UML mapping | Multiplicity & collections | Forward vs reverse workflows | Tool support | Live coding

Where this lecture fits in the course

Builds directly on Lectures 8–11: OO problem solving, composition, design principles, and UML class diagrams.

Lecture 8: identify classes and responsibilities from a problem statement.

Lecture 9: reason about object composition and class design quality.

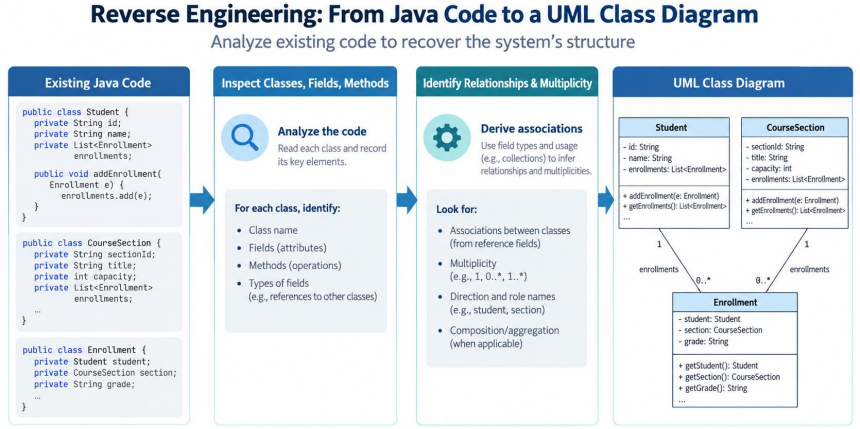
Lecture 10: handle nested composition, multiplicity, and stronger relationship thinking.

Lecture 11: express that design in UML class-diagram notation.

Lecture 12: move in both directions — UML to Java (forward engineering) and Java to UML (reverse engineering).

Why it matters

In practice, software teams rarely live in only one world. Sometimes you start with design and implement it; other times you inherit existing code and must reconstruct the design for understanding, maintenance, or refactoring.



Learning outcomes

By the end of the lecture, students should be able to:

translate core UML class-diagram elements into Java class skeletons;

map multiplicities into references, optional references, and collections;

recognize generalization, interface realization, and dependencies in Java structure;

inspect existing Java code and derive a consistent UML class diagram;

use forward and reverse engineering workflows during live coding and documentation;

avoid common modeling and mapping mistakes.

Teaching style today

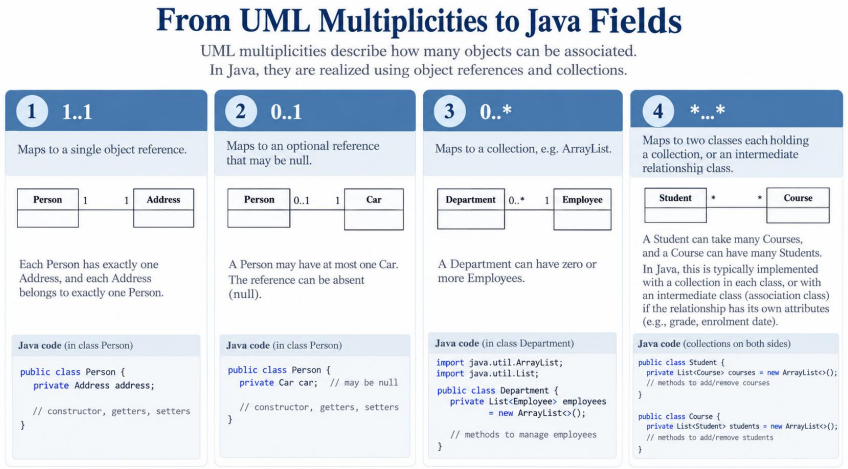
Expect frequent pauses for predict → sketch → code → compile → explain. Keep Command Prompt and your repository folder ready.

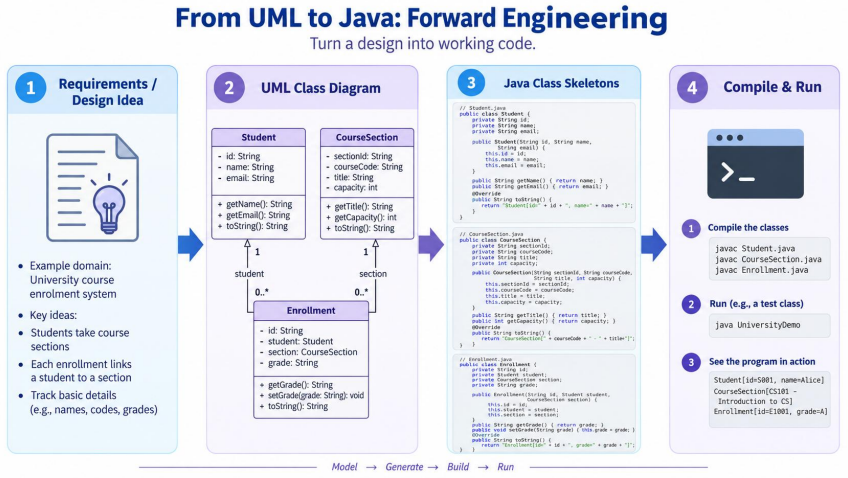
Suggested live-coding path

00_Course_Master\01_Lectures\Week06\L12\LiveCoding\01_ForwardReverseEngineering

Lecture roadmap

1. Conceptual overview: forward engineering vs reverse engineering
2. UML-to-Java mapping rules
3. Worked example: university enrollment mini-system
4. Compile and run the generated code structure
5. Java-to-UML mapping rules
6. Worked example: recover UML from code
7. Tool support, pitfalls, and practice tasks





Forward engineering vs. reverse engineering

Two complementary directions between design and implementation.

Forward engineering

Start from requirements, sketches, or UML models.

Generate or manually write Java class structure.

Useful during initial development and design-driven implementation.

Main question: “How do I implement this design correctly?”

Reverse engineering

Start from existing Java source code.

Recover classes, attributes, operations, and relationships.

Useful for maintenance, onboarding, refactoring, and documentation.

Main question: “What design is already present in the code?”

12

Part I — Forward Engineering

From UML design to Java code structure

Forward engineering workflow

Design ideas are refined into code, then compiled and executed.

From UML Multiplicities to Java Fields

UML multiplicities describe how many objects can be associated.
In Java, they are realized using object references and collections.

1 1..1

Maps to a single object reference.



Each Person has exactly one Address, and each Address belongs to exactly one Person.

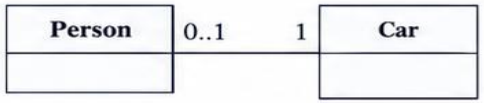
Java code (in class Person)

```
public class Person {
   private Address address;

   // constructor, getters, setters
}
```

2 0..1

Maps to an optional reference that may be null.



A Person may have at most one Car. The reference can be absent (null).

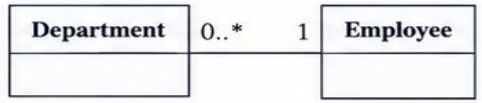
Java code (in class Person)

```
public class Person {
   private Car car;   // may be null

   // constructor, getters, setters
}
```

3 0..*

Maps to a collection, e.g. ArrayList.



A Department can have zero or more Employees.

Java code (in class Department)

```
import java.util.ArrayList;
import java.util.List;

public class Department {
   private List<Employee> employees
           = new ArrayList<>();

   // methods to manage employees
}
```

4 *...*

Maps to two classes each holding a collection, or an intermediate relationship class.



A Student can take many Courses, and a Course can have many Students.

In Java, this is typically implemented with a collection in each class, or with an intermediate class (association class) if the relationship has its own attributes (e.g., grade, enrolment date).

Java code (collections on both sides)

```
public class Student {
  private List<Course> courses = new ArrayList<>();
  // methods to add/remove courses
}

public class Course {
  private List<Student> students = new ArrayList<>();
  // methods to add/remove students
}
```

Systematic UML-to-Java mapping

A class diagram does not write the code for us, but it strongly shapes the code structure.

Class → Java class declaration

Attribute → field / instance variable

Operation → method signature

Visibility (+, -, #, ~) → access modifiers

Multiplicity → single reference, optional reference, or collection

Generalization → extends

Interface realization → implements

Dependency → temporary use in parameters, locals, or return types

Important idea

UML gives a structural blueprint. Java still requires design choices such as constructor design, validation, collection type, and method responsibilities.

Class	→	`class Student`
Attribute	→	`private String name;`
Operation	→	`public String getName()`

Mapping classes to Java declarations

UML class “Student” → Java

```
public class Student {
    // fields
    // constructors
    // methods
}
```

A UML class typically becomes one Java class in its own .java file.

Choose a class name that matches the domain concept and follows Java naming conventions.

If packaging is used, package declaration appears before imports and class declaration.

UML class names are usually singular nouns: Student, CourseSection, Enrollment.

Naming reminder

Classes use PascalCase. File name must match the public class name: Student.java, CourseSection.java, Enrollment.java.

Mapping attributes to fields

Fields from UML attributes

```
private String id;
private String name;
private String email;
private int capacity;
```

Each UML attribute usually becomes a field.

Attribute type maps to a Java type: String, int, double, LocalDate, custom classes, etc.

In an encapsulated design, fields are typically private.

Collection-valued attributes often reflect multiplicity: List<Enrollment>, Set<Course>, Map<String, Student>.

Not every Java field must appear in high-level UML; include only design-relevant state.

Mapping operations to methods

Operations → methods

```
public String getName() {
    return name;
}

public void addEnrollment(Enrollment e) {
    enrollments.add(e);
}
```

Operation name, parameters, and return type map directly to a Java method signature.

UML operation “+ getName(): String” becomes “public String getName()”.

Void-returning operations map to methods with return type void.

Method bodies are not shown in class diagrams but must be designed in code.

Method responsibilities should align with class responsibilities.

Mapping visibility and access control

UML	Meaning	Typical Java mapping
+	public	Accessible everywhere.
-	private	Accessible only inside the same class.
#	protected	Accessible in the same package and in subclasses.
~	package/default	No keyword; accessible inside the same package.

Teaching recommendation

When in doubt, begin with private fields and expose behavior through methods. This keeps the model clean and preserves invariants.

Mapping creation to constructors

Constructors are not always emphasized in basic UML, but they are essential in Java implementation.

A diagram may imply that certain fields must be initialized at object creation time.

Constructors establish valid initial state and enforce invariants.

No-argument, parameterized, and copy-style constructors are code-level design decisions.

For a many-valued relationship, initialize a suitable array/collection and enforce any capacity or uniqueness rules.

Example constructor

```
public Student(String id, String name, String
    this.id = id;
    this.name = name;
    this.email = email;
    this.enrollments = new ArrayList<>();
}
```

Reminder

Forward engineering is not mechanical translation alone. You must still decide what constructors and validations best serve the class design.

Mapping multiplicity into Java

1 **Exactly one**

```
private Student student;
```

0..1 **Zero or one**

```
private Room room;
```

0..* **Zero or more**

```
Enrollment[] records;
```

1..* **One or more**

```
Nonempty array + validation
```

Multiplicity describes allowed object links; arrays and fields only implement them. Validity requires constructor/method checks.



Model → Generate → Build → Run

Association, aggregation, and composition in code

Java syntax alone cannot fully express the semantic strength of a relationship; design intent matters.

Common code patterns

Association: instances maintain a structural link; a field can represent this link.

Aggregation: one class stores references to objects that may outlive it.

Composition: one class strongly owns objects whose lifetime depends on it.

Collections are common for one-to-many relationships.

Typical cues in code

Field of another class: private Student student;

Collection of other objects: List<Enrollment> enrollments;

Construction inside owner: this.schedule = new MeetingSchedule(...);

Creation location alone does not prove composition; ownership and lifecycle must follow domain rules.

Mapping inheritance and interfaces

Generalization and interface realization

```
public abstract class Person {
    private String id;
    public abstract String getRole();
}

public class Student extends Person {
    @Override
    public String getRole() { return "Student"; }
}

public interface Reportable {
    String generateReport();
}

public class CourseSection implements Reportable {
    public String generateReport() { return "..."; }
}
```

UML generalization arrow → Java extends.

UML interface realization → Java implements.

Abstract classes and abstract operations should map cleanly to abstract classes/methods in Java.

Forward engineering must also consider overriding, visibility, and constructor chaining.

Mapping dependencies vs. long-term associations

Dependency (“uses-a”) example

```
public class EnrollmentService {
    public void printStudent(Student s) {
        System.out.println(s);
    }
}
```

Association example

```
public class Enrollment {
    private Student student;
    private CourseSection section;
}
```

If another class is used only inside a method parameter, local variable, or return type, UML may show a dependency rather than a stored association.

If the class keeps a field reference, the relationship is stronger and usually modeled as an association.

Reverse engineering must separate temporary use from stored structure.

Live coding plan for forward engineering

Create Student, CourseSection, and Enrollment class files.

Add fields based on the UML attributes.

Add constructors, getters, setters (where needed), and toString().

Create UniversityDemo to instantiate objects and print them.

Compile with javac and run with java.

Discuss what design choices came from the diagram and what choices came from the programmer.

Command Prompt workflow

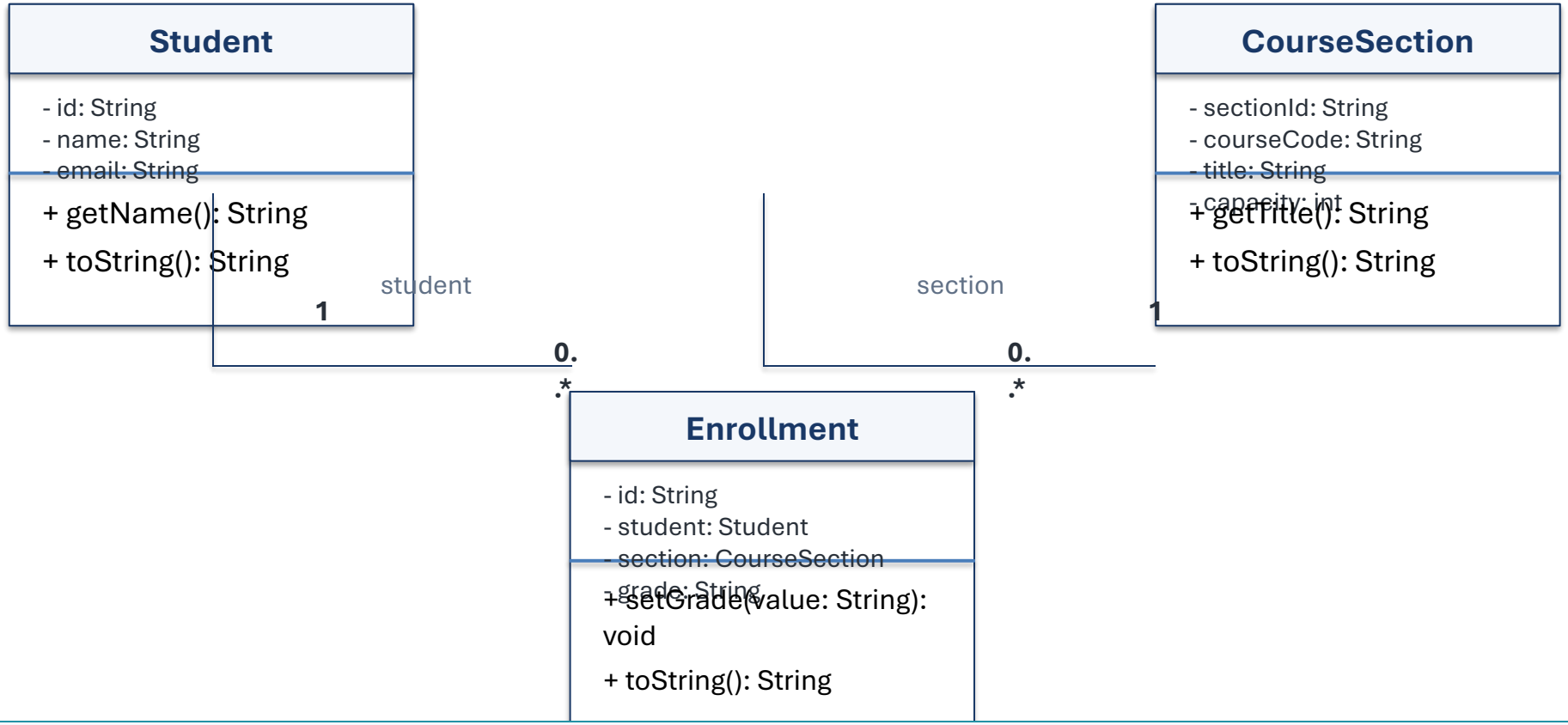
```
javac Student.java
javac CourseSection.java
javac Enrollment.java
javac UniversityDemo.java
java UniversityDemo
```

Repository suggestion

Students can store today's example in a folder such as Lecture12_ForwardReverseEngineering inside their lab repository.

Worked example — conceptual model

Mini domain: university course enrollment



Interpretation

One Student can participate in many Enrollment objects. One CourseSection can also appear in many Enrollment objects. Each Enrollment links exactly one Student with exactly one CourseSection.

Forward engineering example — Student.java

Student.java

```
public class Student {
    private String id, name, email;
    private Enrollment[] enrollments = new Enrollment[10];
    private int count = 0;

    public Student(String id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }
    public String getName() { return name; }
    public void addEnrollment(Enrollment e) {
        if (count == enrollments.length)
            throw new IllegalStateException("Full");
        enrollments[count++] = e;
    }
    public String toString() {
        return "Student[id=" + id + ", name=" + name + "]";
    }
}
```

- Uses an array of references: students already know arrays.
- The counter tracks occupied slots.
- A full array requires a capacity check.
- toString() makes the driver output meaningful.
- 0..* does not mean a Java array grows automatically.

Forward engineering example — CourseSection.java

CourseSection.java

```
public class CourseSection {
    private String sectionId, courseCode, title;
    private int capacity;

    public CourseSection(String sectionId,
                         String courseCode,
                         String title, int capacity) {
        this.sectionId = sectionId;
        this.courseCode = courseCode;
        this.title = title;
        this.capacity = capacity;
    }
    public String getTitle() { return title; }
    public String toString() {
        return "CourseSection[" + courseCode
               + " - " + title + "]";
    }
}
```

- Attributes map to instance fields.
- Constructor supplies essential values.
- getTitle() maps from the UML operation.
- toString() matches the displayed output.
- The UML shows allowable relationships; it does not require both ends to store duplicate arrays.

Forward engineering example — Enrollment.java

Enrollment.java

```
public class Enrollment {
    private String id;
    private Student student;
    private CourseSection section;
    private String grade = "Not Assigned";

    public Enrollment(String id, Student student,
                      CourseSection section) {
        if (student == null || section == null)
            throw new IllegalArgumentException();
        this.id = id;
        this.student = student;
        this.section = section;
    }
    public void setGrade(String grade) {
        this.grade = grade;
    }
    public String toString() {
        return "Enrollment[id=" + id
               + ", grade=" + grade + "]";
    }
}
```

This class is often called an association class because the relationship itself carries data (grade).

Fields student and section embody the links implied by the diagram.

Single fields implement one related Student and one CourseSection; null checks enforce required links.

Teaching insight

A direct many-to-many between Student and CourseSection would lose relationship-specific data such as grade or enrollment date. Introducing Enrollment produces a richer model.

Driver class, compilation, and program output

UniversityDemo.java

```
public class UniversityDemo {
    public static void main(String[] args) {
        Student s = new Student("S001", "Abeer",
                                "abeer@cui.com");
        CourseSection cs = new CourseSection(
            "L12-A", "CSC241",
            "Object-Oriented Programming", 40);
        Enrollment e = new Enrollment("E001", s, cs);
        e.setGrade("A-");
        s.addEnrollment(e);
        System.out.println(s);
        System.out.println(cs);
        System.out.println(e);
    }
}
```

Command Prompt

```
javac *.java
java UniversityDemo
```

Sample output

```
Student[id=S001, name=Abeer]
CourseSection[CSC241 - Object-Oriented Programming]
Enrollment[id=E001, grade=A-]
```

What came from UML, and what came from the programmer?

Driven by the diagram

Class names and major responsibilities

Core attributes

Relationship directions and multiplicities

Inheritance / interface structure (if present)

Key message

Forward engineering is guided translation, not blind code generation.

Chosen in Java implementation

Constructor signatures and validations

Collection type (ArrayList, HashSet, etc.)

Exact method bodies

Utility methods such as toString(), equals(), hashCode()

12

Part II — Reverse Engineering

From existing Java code back to a class diagram

Reverse engineering workflow

Read the code, discover its structure, then recover a design view.

Reverse Engineering: From Java Code to a UML Class Diagram

Analyze existing code to recover the system's structure

Existing Java Code

```
public class Student {
   private String id;
   private String name;
   private List<Enrollment>
           enrollments;

   public void addEnrollment(
      Enrollment e) {
         enrollments.add(e);
   }
}
```

```
public class CourseSection {
   private String sectionId;
   private String title;
   private int capacity;
   private List<Enrollment>
           enrollments;
   …
}
```

```
public class Enrollment {
   private Student student;
   private CourseSection section;
   private String grade;
   …
}
```

Inspect Classes, Fields, Methods

Analyze the code

Read each class and record its key elements.

For each class, identify:

- Class name
- Fields (attributes)
- Methods (operations)
- Types of fields (e.g., references to other classes)

Identify Relationships & Multiplicity

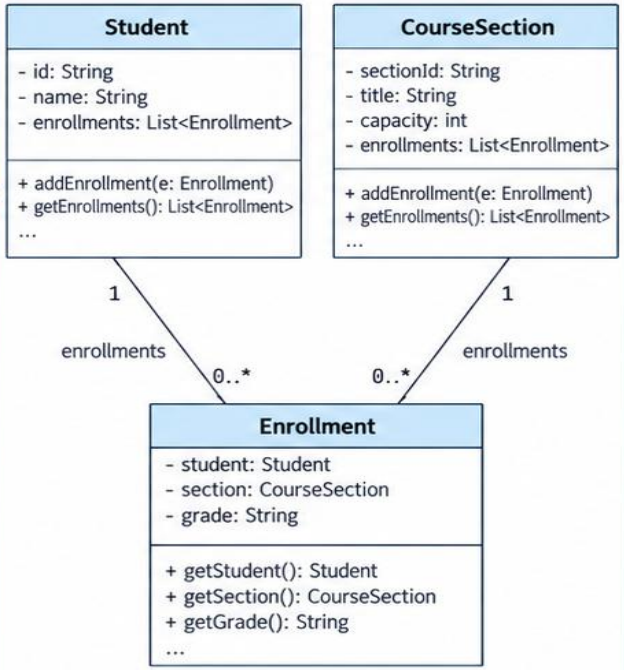
Derive associations

Use field types and usage (e.g., collections) to infer relationships and multiplicities.

Look for:

- Associations between classes (from reference fields)
- Multiplicity (e.g., 1, 0..*, 1..*)
- Direction and role names (e.g., student, section)
- Composition/aggregation (when applicable)

UML Class Diagram



Why do we reverse engineer code?

to understand an unfamiliar codebase quickly;

to create or update technical documentation;

to detect poor structure before refactoring;

to visualize relationships that are hard to see from many source files;

to support onboarding, maintenance, testing, and redesign.

Real classroom connection

Students often write code first in labs. Reverse engineering helps them step back and explain what design they actually produced.

Systematic Java-to-UML mapping process

Read all classes and list their names.

For each class, note fields, constructors, and public operations.

Mark visibility: private, public, protected, or package-private.

Identify field types referencing other classes.

Look for collections to infer one-to-many or many-to-many relationships.

Distinguish temporary method usage from stored long-term references.

Add inheritance and interface relationships.

Refine multiplicity, role names, and dependency arrows.

Tip for students

Start rough. First draw the classes and main fields. Then add details in a second pass. Perfect UML is rarely produced in one attempt.

What should you inspect in the code?

Class-level clues

class / abstract class / interface

extends / implements

package, imports

Field-level clues

private Student student;

List<Enrollment> enrollments;

static/final members

Method-level clues

parameters and return types

constructors

methods that use but do not store another class

Inferring multiplicity from Java code

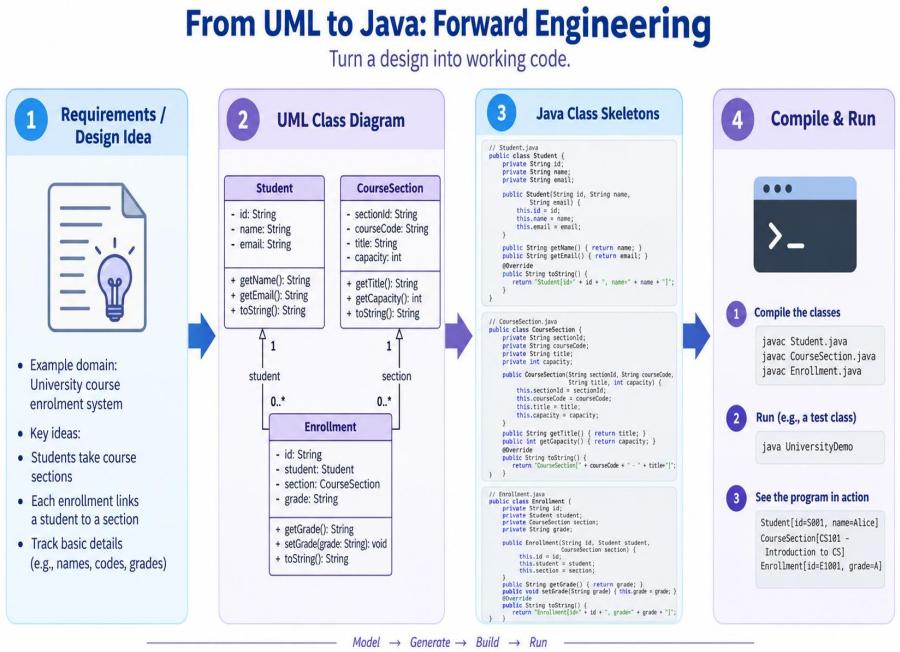
Student student; → one field; may represent 0..1 or 1

Car car; → often 0..1 or 1 depending on domain rules

Enrollment[] enrollments; → bounded storage; logical count is separate

List<Course> courses; → many links; collection semantics vary

Two classes holding collections may indicate many-to-many or a missing association class



Important warning

Multiplicity is inferred from both code and domain meaning. A single reference does not tell us by itself whether the intended multiplicity is 1 or 0..1. Domain rules decide that.

Inferring inheritance, interfaces, and dependencies

Code clues

```
public class Student extends Person implements Reportable {
    @Override
    public String generateReport() { ...
}
```

extends → UML generalization (inheritance)

implements → UML realization from class to interface

method parameters / local variables / return types without stored fields → UML dependency

stored fields → stronger association or aggregation/composition

Example distinction

If EnrollmentPrinter.print(Student s) only prints a Student, that is a dependency. If Enrollment stores a Student field, that is an association.

Worked reverse-engineering example

Given code fragments

```
public class Book {
    private String isbn;
    private String title;
}

public class Member {
    private String id;
    private String name;
    private List<Loan> loans = new ArrayList<>();
}

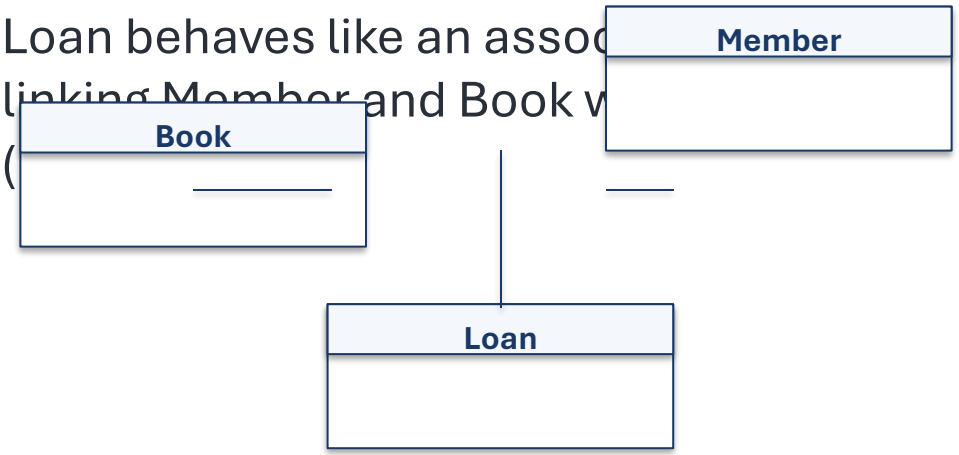
public class Loan {
    private Book book;
    private Member member;
    private LocalDate issueDate;
}
```

Classes discovered: Book, Member, Loan.

Loan stores a Book reference and a Member reference (requiredness needs validation).

Member holds many Loan objects.

Loan behaves like an assoc
linking Member and Book w
(



Common tooling options

Examples of tools commonly used to support design-to-code and code-to-design workflows

PlantUML — text-based UML diagrams that integrate well with version control.

StarUML / Visual Paradigm — dedicated modeling tools for drawing and sometimes generating code skeletons.

IntelliJ IDEA UML/class-diagram functionality depends on edition and plugin availability; verify your installation.

Lucidchart / draw.io — convenient for manually documenting class diagrams.

Even without tools, systematic sketching on a whiteboard remains valuable for reasoning.

Balanced advice

Do not over-depend on one-click generation. Good engineers still verify whether the generated model or code reflects meaningful responsibilities and relationships.

Common mistakes and misconceptions

Treating every class reference as composition; sometimes it is only association or dependency.

Assuming a single field automatically means multiplicity 1 rather than 0..1.

Ignoring collection fields when drawing reverse-engineered UML.

Forgetting to map interfaces, abstract classes, or overridden behavior.

Generating code that mirrors the diagram structurally but has weak behavior and poor responsibility assignment.

Omitting an association class when the relationship carries attributes (grade/date).

Teacher's checkpoint question

Ask: "What information does the diagram force, and what design choices still remain open?" Students who can answer this are thinking beyond syntax.

In-class activity (15–20 minutes)

Sketch first, then verify in code.

Activity A: From the Student–Enrollment–CourseSection UML (slide 19), write the Java class skeletons in pairs.

Activity B: Reverse-engineer Book, Member, Loan (slide 32) into a UML diagram on paper.

Activity C: Explain whether a relationship is dependency, association, aggregation, or composition, and justify your choice.

Activity D: Decide whether a direct many-to-many link should be replaced with an association class.

Possible class domain ideas

Library loaning, hospital appointments, hostel room allotment, shopping cart, lab equipment booking.

Deliverable during class

One sketched diagram + one compiled Java example + a short explanation of two mapping decisions.

Summary and follow-up

Forward engineering converts UML design ideas into Java class structure.

Reverse engineering extracts design knowledge from existing code.

Multiplicity, inheritance, interfaces, and dependencies require careful interpretation.

UML and code should reinforce each other rather than drift apart.

A strong engineer can comfortably move in both directions.

Suggested practice

1) Complete the university example. 2) Add toString() methods. 3) Draw the UML of your final code. 4) Commit the solution to your GitHub repository.

Next lecture bridge

Lecture 13 will deepen decomposition and class hierarchy design, where these modeling-to-code skills become even more important.